

Seeing the Forest and the Trees: Multi-Scale Voting Improves Test-Time GUI Grounding

Jian Liang¹, Kuangpu Guo^{1,2}, Yongcan Yu¹, and Lijun Sheng^{3*}

¹ NLPR & MAIS, Institute of Automation, Chinese Academy of Sciences

² University of Science and Technology of China ³ JD.COM

liangjian92@gmail.com

Abstract. GUI grounding, the localization of screen elements based on natural language instructions, is essential for autonomous GUI agents. Supervised training achieves strong accuracy but requires massive labeled data, making annotation expensive and limiting generalization to new environments. Recent work addresses this by exploring test-time grounding, which exploits unlabeled data available during inference. A representative approach in this vein is test-time scaling, which performs spatial voting across multiple sampled outputs to deliver a more stable and accurate prediction. In this paper, we identify scale variation as an important yet overlooked factor in test-time GUI grounding and propose GUI-SV, a simple multi-scale spatial voting method. Specifically, GUI-SV scales the full image up and down, produces multiple predictions per scale, and aggregates them via spatial voting to produce the final output. We further present GUI-SVPO, a test-time reinforcement learning approach that uses pseudo-labels from GUI-SV as proxy rewards to refine the model without human annotations. Extensive experiments across three datasets and various backbones confirm that GUI-SV achieves state-of-the-art performance. Its training variant, GUI-SVPO-SV, further boosts the accuracy on ScreenSpot-Pro from 20.87% to 33.27%. Despite its simplicity, we believe that this work provides a promising path toward label-efficient GUI agents.

Keywords: GUI grounding · Test-time scaling · Data augmentation

1 Introduction

Graphical user interfaces (GUIs) are the primary medium through which users interact with digital devices, comprising elements such as buttons, menus, and text fields. As autonomous agents are increasingly used to perform tasks on behalf of users [8, 32], they must accurately map a natural language instruction to the corresponding element on the screen before executing any action. This capability is known as GUI grounding. It is a foundational component of GUI agents: errors at this stage propagate directly to downstream planning and execution.

* Corresponding author.

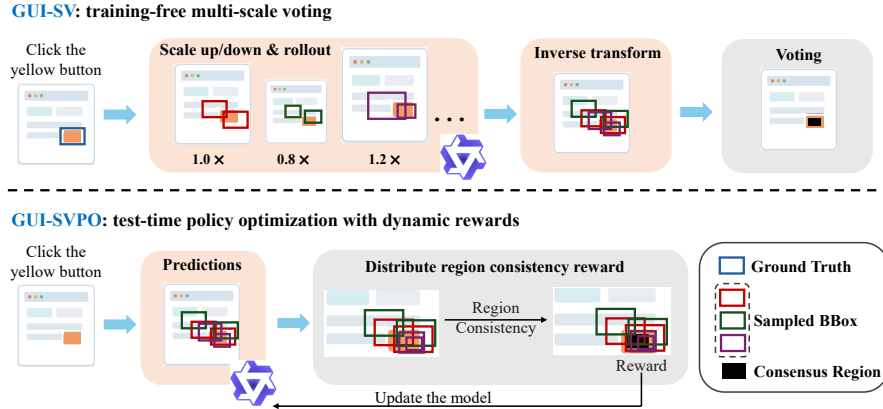


Fig. 1: Pipeline of the proposed GUI-SV (top) and its test-time reinforcement learning variant, GUI-SVPO (bottom). The core idea is to leverage multiple scales across the entire image before prediction rollouts. GUI-SVPO then uses the voted prediction from GUI-SV to update the model via policy optimization.

Existing GUI grounding approaches fall into two paradigms. The first formulates grounding as point prediction, where the model directly outputs the target coordinate, as in SeeClick [4] and UGround [6]. The second predicts a bounding box representing the target region, as in OS-Atlas [36] and GUI-Actor [35]. Point-based prediction integrates seamlessly with downstream actions but often fails to capture the spatial ambiguity inherent in human-GUI interaction, which has motivated box-based alternatives. Both paradigms, however, achieve strong performance primarily through extensive labeled data. Some methods rely on supervised fine-tuning, directly learning to predict coordinates or bounding boxes from annotated pairs [15, 25]. Others use reinforcement learning with reward signals computed from ground-truth labels [16, 18, 19]. Either way, labeled data is indispensable. This reliance limits their applicability in label-scarce scenarios, where large-scale GUI annotations are costly to collect.

To reduce this dependence, test-time scaling [21, 28, 41] has recently gained popularity. It aims to improve inference-time performance through additional computation rather than extra labels. A representative example is test-time reinforcement learning (TTRL) [46], which generates reward signals without ground-truth labels by repeatedly sampling model outputs and deriving consensus via majority voting. This simple strategy boosts Qwen-2.5-Math-7B by an average of 76% across different datasets, using only unlabeled test data.

Inspired by this idea, a prior work [5] brings test-time RL to GUI grounding. It also samples multiple predictions in the form of points or boxes and aggregates them spatially. However, this approach operates at a single scale, where every prediction comes from the same fixed view of the screenshot. This overlooks a key property of GUI screenshots: target elements vary drastically in size, from tiny icons on high-resolution professional interfaces to large buttons on mobile

screens. At a single scale, the model’s errors are often correlated rather than diverse. A small icon may be consistently mislocalized simply because it loses resolution once the full screenshot is encoded, and voting over such correlated errors reinforces the mistake rather than correcting it.

To address this, we propose GUI-SV, a simple yet effective multi-scale spatial voting method (Fig. 1). It applies up- and down-scaling to the full screenshot to generate multiple views at different resolutions, then samples predictions from each view and aggregates them via spatial voting. Unlike prior cropping-based approaches, we resize the entire image. This design preserves the complete spatial context across all views, ensuring that the model perceives the full layout and avoids excluding the target element. We further introduce a TTRL variant of GUI-SV, dubbed GUI-SVPO, which enables model updates during the test phase. Specifically, for each unlabeled sample, GUI-SVPO leverages the training-free GUI-SV method to obtain a consensus region as a pseudo-label. During the rollout process, rewards are computed based on the alignment between each prediction and this pseudo-label. Finally, we evaluate GUI-SV on the model updated by GUI-SVPO and observe consistent performance gains.

2 Related Works

GUI grounding is a foundational capability for GUI agents, enabling the translation of natural-language instructions into executable interface actions such as clicking, typing, and navigation. Early approaches, including SeeClick [4] and concurrent efforts [7, 9, 33, 36, 38, 43], directly trained visual models to locate instruction-relevant UI elements from full screenshots. Subsequent work has advanced this paradigm along two fronts: data scaling and agent modeling. UGround [24] leverages large-scale synthetic data to boost visual grounding; OS-Atlas [36] builds a comprehensive cross-platform grounding corpus; and ShowUI [14] introduces UI-aware visual token selection to better align vision, language, and action. More recent end-to-end agents and grounding models such as UI-TARS [25, 31], MAI-UI [44], GUI-Owl [37], and Aria-UI [39] further integrate grounding into native or pure-vision architectures.

From alternative optimization perspectives, UI-Ins [3] enhances instruction-level reasoning, and GUI-Actor [35] replaces direct coordinate generation with action-region prediction. Meanwhile, SE-GUI [40], GUI-G2 [30], GUI-RCPO [5], InfiGUI-G1 [15], and GuirLVG [11] refine training and inference through dense spatial rewards, region consistency, reinforcement learning exploration, or stabilized fine-tuning. Despite these advances, most existing methods overlook a fundamental mismatch: the generative MLLM paradigm is ill-suited to the fine-grained spatial alignment demanded by point-level GUI grounding.

To address this, several recent works adopt zoom-based strategies that revisit local regions at higher effective resolution. RegionFocus [20] formalizes dynamic zoom-in as visual test-time scaling; ZoomClick [10] studies zooming as a training-free prior; and DiMo-GUI [34] combines dynamic visual grounding with modality-aware reasoning over textual and iconic elements. More adap-

tive frameworks include UI-Zoomer [29], which triggers zoom-in based on prediction uncertainty; AdaZoom-GUI [23], which couples instruction refinement with conditional zooming; and ZoomUI [17], which progressively anchors instructions through latent thinking and attention-guided zooming. Beyond zooming, MVP [42] aggregates predictions from multiple attention-guided views to stabilize coordinate outputs, while Chain-of-Ground [12] and Iterative Narrowing [22] refine grounding via multi-step reasoning or progressive cropping. GUI-Eyes [2] introduces tool-augmented focus refinement and active visual perception. However, none of these approaches explore whole-image rescaling as a data augmentation technique to improve grounding robustness.

3 Method

Following the previous work [5], we use multiple grounding predictions and spatial voting to extract a consensus region. Our methods extend this procedure by introducing multiple resized views of the input screenshot and mapping all predictions back to the original coordinate system.

Let I denote a screenshot with width W and height H , and let u denote the corresponding instruction. We use K as the total sampling number. Given N augmented views, the base number of samples for each view is

$$q = \left\lfloor \frac{K}{N+1} \right\rfloor, \quad r = K - q(N+1), \quad (1)$$

where each augmented view receives q samples, and the original image receives the remaining r samples.

3.1 GUI-SV: Multi-scale Spatial Voting

Multi-scale image construction. In the training-free setting, GUI-SV generates multiple views by isotropically resizing the original screenshot. For a scale factor s , the resized image is defined as

$$I^{(s)} = \text{Resize}(I, \lfloor sH \rfloor, \lfloor sW \rfloor), \quad (2)$$

where the height and width are scaled by the same factor, $s_x = s_y = s$. The original screenshot is included as the view with $s = 1$. Let (x, y) be a point in the original image coordinate system. Its coordinate in the resized image is

$$(x', y') = (sx, sy). \quad (3)$$

Therefore, a prediction (x', y') obtained from the resized image is mapped back to the original image by

$$(x, y) = \left(\frac{x'}{s}, \frac{y'}{s} \right). \quad (4)$$

Similarly, a bounding box $b = [x_1, y_1, x_2, y_2]$ is transformed into $b^{(s)} = [sx_1, sy_1, sx_2, sy_2]$, and the inverse mapping is

$$b = \left[\frac{x'_1}{s}, \frac{y'_1}{s}, \frac{x'_2}{s}, \frac{y'_2}{s} \right]. \quad (5)$$

Point-style predictions are first mapped back to the original coordinates and expanded into a square region of side length α :

$$r_k = \left[x_k - \frac{\alpha}{2}, y_k - \frac{\alpha}{2}, x_k + \frac{\alpha}{2}, y_k + \frac{\alpha}{2} \right]. \quad (6)$$

For bounding-box models, the inversely mapped box is directly taken as r_k . All K predictions from the original and resized views are then represented in the original image coordinate system. We follow the previous work to construct a voting grid and extract the consensus region:

$$v(x, y) = \sum_{k=1}^K \mathbb{I}[(x, y) \in r_k]. \quad (7)$$

The final prediction is obtained by selecting the largest connected component with the maximum vote value. We refer to this training-free method as GUI-SV.

3.2 GUI-SVPO: Policy Optimization with GUI-SV Rewards

GUI-SVPO extends the previous policy optimization method by introducing multi-scale augmentation into the rollout process. Unlike GUI-SV, the width and height are scaled independently to introduce stronger geometric variations. We then sample predictions from all views using the same sampling allocation and decoding settings as GUI-SV. We aggregate the inverse-mapped rollout predictions and construct the voting grid using the same procedure as GUI-SV. The consensus region extracted from the current rollout serves as a dynamic pseudo-label.

Following the previous work [5], the region consistency reward for the k -th mapped prediction r_k is

$$R_{\text{rc}}(k) = \frac{1}{|r_k|v_{\max}} \sum_{(x,y) \in r_k} v(x, y), \quad (8)$$

where $v(x, y)$ is the voting value at position (x, y) and

$$v_{\max} = \max_{(x,y)} v(x, y). \quad (9)$$

Predictions that overlap with high-consistency regions receive larger rewards, whereas spatially inconsistent predictions receive smaller rewards. The voting result and rewards are dynamically computed from the same rollout and immediately used for the current optimization step. Finally, we use Group Relative Policy Optimization (GRPO) to optimize the VLM with the region consistency rewards. The GRPO objective and optimization procedure remain unchanged.

4 Experiments

4.1 Setup

Models. Our test-time scaling strategy is flexible across diverse output representations, including both point-based coordinates and region-based bounding boxes. To demonstrate this generality, we choose six SOTA vision-language architectures covering diverse pre-training paradigms. For general multimodal understanding, we incorporate Qwen2.5-VL (3B and 7B variants) [1] and InternVL3-8B-Instruct [45]. To further assess performance on domain-aligned agents, we additionally evaluate GUI-centric models explicitly optimized for screen grounding: UGround-V1-7B [6], UI-TARS-1.5-7B [26], and GUI-G2-7B [30].

Evaluation benchmarks and metrics. We quantify grounding performance using standard Grounding Accuracy [4], where a predicted coordinate or the centroid of a predicted bounding box is considered correct if it falls strictly within the target ground-truth box. We evaluate across three mainstream GUI benchmarks of varying complexity. ScreenSpot [4] and ScreenSpot-v2 [36] assess broad multi-platform capabilities across Mobile, Web, and Desktop interfaces, while ScreenSpot-Pro [13] serves as a more challenging testbed, stress-testing our approach on fine-grained, high-resolution professional application layouts.

Implementation details. For our training-free method, GUI-SV, each query image is augmented to three scales using factors of 0.8, 1.0 (original), and 1.2. We generate 64 candidate predictions per query, distributed as evenly as possible across the three scale views, with any remainder allocated to the original image. Sampling is performed with a temperature of 0.5, and the hyperparameter α is set to 50. For our TTRL variant, GUI-SVPO, we build upon the VLM-R1 [27] framework and conduct training without ground-truth annotations. Each instance uses the same multi-scale augmentation and 64-rollout scheme as GUI-SV, but with a sampling temperature of 0.7. We optimize the model for 2 epochs (~ 40 steps) with a global batch size of 64, a learning rate of 1×10^{-7} , and a KL penalty factor $\beta = 0.04$. All training and evaluation are conducted on 8 NVIDIA A100-80GB GPUs.

4.2 Main Results

GUI-SV consistently boosts grounding across diverse models and benchmarks. We first evaluate the performance of our training-free method, GUI-SV, across six baseline VLMs on ScreenSpot, ScreenSpot-v2, and ScreenSpot-Pro. As shown in Table 1 and Table 2, GUI-SV achieves consistent and substantial accuracy gains over both the base models and the baseline GUI-RC [5] across almost all testing scenarios. For instance, on ScreenSpot-v2, GUI-SV elevates Qwen2.5-VL-3B-Instruct from 80.27% to 84.67%. Notably, GUI-RC sometimes leads to performance drops on certain architectures, while GUI-SV exhibits strong robustness across all.

Evaluation of GUI-SVPO. We further evaluate our test-time reinforcement learning variant, GUI-SVPO, which utilizes multi-scale visual self-supervision

Table 1: Performance comparison for training-free methods on ScreenSpot-Pro, which features higher-resolution interfaces and greater domain shift (e.g., industrial software, multi-window layouts), serving as a practical testbed for generalization.

Model	Dev	CAD	Creative	Scientific	Office	OS	Avg-Text	Avg-Icon	Avg.
<i>General Models</i>									
InternVL3-8B-Instruct	8.21	4.35	10.26	13.62	19.57	7.14	15.51	2.14	10.40
w/ GUI-RC	7.69	3.45	6.16	17.71	17.83	7.14	14.23	2.32	9.68
w/ GUI-SV	10.70	4.21	12.17	16.53	16.96	7.65	16.98	2.42	11.42
Qwen2.5-VL-3B-Instruct	12.04	20.31	21.41	28.74	29.13	14.28	30.30	5.63	20.87
w/ GUI-RC	14.38	22.60	24.34	30.32	39.13	14.80	33.78	8.44	24.10
w/ GUI-SV	18.40	24.14	28.15	35.04	43.48	18.88	39.30	9.27	27.83
Qwen2.5-VL-7B-Instruct	17.39	11.11	16.13	25.98	35.65	23.47	29.38	7.12	20.87
w/ GUI-RC	23.41	12.26	19.35	26.38	37.39	24.49	32.96	7.78	23.34
w/ GUI-SV	29.43	13.03	19.94	28.35	46.09	26.53	37.16	9.44	26.57
<i>GUI-specific Models</i>									
UGround-V1-7B	31.44	13.80	29.91	38.98	52.18	23.98	46.47	7.28	31.50
w/ GUI-RC	28.76	13.79	27.57	35.83	44.78	20.92	41.96	6.79	28.53
w/ GUI-SV	31.77	13.79	29.33	36.22	48.26	23.98	45.14	6.62	30.42
UI-TARS-1.5-7B	32.58	30.65	35.50	50.06	62.49	31.93	54.48	16.50	39.97
w/ GUI-RC	33.78	33.33	36.37	46.85	61.74	29.08	55.78	14.07	39.85
w/ GUI-SV	37.46	32.18	37.83	45.67	58.26	32.65	54.76	17.22	40.42
GUI-G2-7B	36.12	46.36	40.18	52.76	66.09	34.19	60.70	20.86	45.48
w/ GUI-RC	37.46	44.06	38.71	55.11	65.65	32.66	60.39	20.53	45.16
w/ GUI-SV	52.18	52.49	44.57	58.27	73.04	39.79	69.29	26.82	53.07

to guide GRPO optimization without ground-truth labels. Table 3 presents the results when evaluating the trained models in a zero-shot inference manner without test-time augmentation. Across all evaluated backbones, GUI-SVPO consistently surpasses the baseline GUI-RCPO [5]. For instance, on ScreenSpot-v2, GUI-SVPO achieves 87.42% and 90.09% accuracy on Qwen2.5-VL-3B and 7B, outperforming GUI-RCPO, which reaches 87.11% and 89.31%, respectively. This confirms that multi-scale visual consistency serves as a superior self-supervised reward generator during RL policy optimization compared to pure decoding reward temperature variation.

Complementary gains with test-time scaling (GUI-SVPO-SV). Finally, we investigate the joint effect of multi-scale TTRL training and multi-scale test-time scaling by applying GUI-SV during inference on the GUI-SVPO trained models, denoted as GUI-SVPO-SV. As demonstrated in Table 3, GUI-SVPO-SV yields the overall best grounding performance across all settings, substantially outperforming GUI-RCPO-RC [5]. On ScreenSpot-Pro, GUI-SVPO-SV achieves a staggering 33.27% accuracy for Qwen2.5-VL-3B-Instruct and reaches 46.00% for UI-TARS-1.5-7B. These results demonstrate that multi-scale TTRL training and test-time visual scaling are mutually beneficial, pushing the boundaries of self-supervised GUI grounding.

Table 2: Performance comparison for training-free methods on ScreenSpot and ScreenSpot-v2. Detailed Mobile/Desktop/Web (Text/Icon) results are reported on ScreenSpot. For ScreenSpot-v2, we report the overall average only.

Model	Mobile		Desktop		Web		SS.Avg	SSv2.Avg
	Text	Icon	Text	Icon	Text	Icon		
General Models								
InternVL3-8B-Instruct	93.77	81.66	79.38	59.29	86.52	69.42	80.35	81.44
w/ GUI-RC	93.41	79.91	80.93	61.43	85.65	70.39	80.43	82.71
w/ GUI-SV	93.41	82.10	80.41	57.86	86.52	70.39	80.50	82.39
Qwen2.5-VL-3B-Instruct	95.97	68.56	86.60	55.71	80.43	63.59	77.12	80.49
w/ GUI-RC	96.70	72.49	88.66	60.00	83.48	63.59	79.32	82.07
w/ GUI-SV	95.60	72.05	90.21	63.57	85.65	71.36	81.29	84.67
Qwen2.5-VL-7B-Instruct	95.60	79.91	85.57	72.86	85.65	75.73	83.73	86.56
w/ GUI-RC	96.34	81.22	91.75	72.86	88.70	77.18	85.85	88.44
w/ GUI-SV	96.70	82.97	92.27	77.14	89.13	76.70	86.79	89.70
GUI-specific Models								
UGround-V1-7B	94.51	78.60	93.81	75.00	90.43	77.67	85.93	88.52
w/ GUI-RC	94.51	79.91	93.81	74.29	90.43	79.61	86.40	88.21
w/ GUI-SV	95.60	79.91	94.33	72.14	91.30	78.16	86.40	88.29
UI-TARS-1.5-7B	94.14	81.22	94.85	78.57	88.70	85.44	87.82	90.49
w/ GUI-RC	93.77	83.41	93.30	82.86	90.00	86.41	88.76	91.04
w/ GUI-SV	95.60	86.46	94.85	85.00	91.74	87.38	90.64	92.85
GUI-G2-7B	96.70	89.96	94.33	84.29	90.87	87.38	91.20	93.39
w/ GUI-RC	96.70	89.96	94.33	85.71	91.30	86.41	91.27	93.55
w/ GUI-SV	97.07	90.83	95.88	88.57	91.74	88.35	92.45	94.42

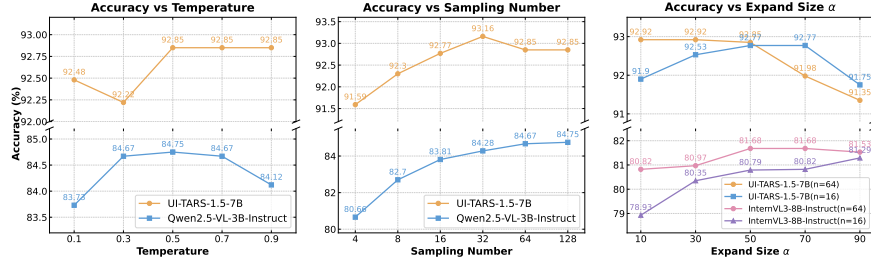


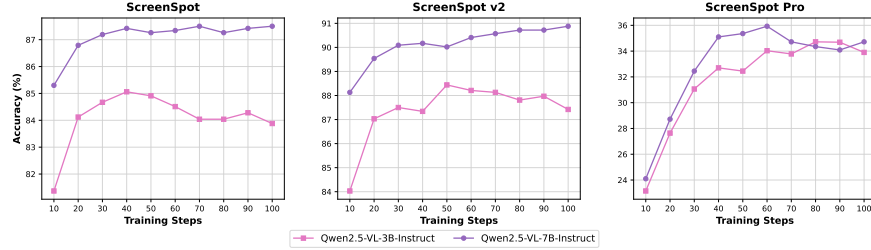
Fig. 2: Ablation study results on ScreenSpot-v2 with varying temperature, sampling number, and hyperparameter α .

4.3 Ablation Studies

To systematically evaluate the components of GUI-SV, we conduct ablation experiments on the ScreenSpot-v2 benchmark. We investigate the model’s sensitivity to key inference hyperparameters (sampling temperature, rollout budget N , and expansion size α) alongside different multi-scale visual augmentation configurations.

Table 3: Results of training-based variants and their augmented versions across ScreenSpot, ScreenSpot-v2, and ScreenSpot-Pro.

Model	SS			SSv2			SSpro		
	Text Avg.	Icon Avg.	Avg.	Text Avg.	Icon Avg.	Avg.	Text Avg.	Icon Avg.	Avg.
<i>General Models</i>									
Qwen2.5-VL-3B-Instruct	88.23	63.65	77.12	90.78	67.15	80.49	30.30	5.63	20.87
w/ GUI-RCPO	92.68	73.22	83.88	95.40	76.36	87.11	41.25	8.78	28.84
w/ GUI-SVPO	93.26	74.09	84.59	95.82	76.54	87.42	41.45	9.44	29.22
w/ GUI-RCPO-RC	92.97	73.39	84.12	95.68	76.71	87.42	40.84	10.27	29.16
w/ GUI-SVPO-SV	93.97	76.35	86.01	96.80	79.60	89.31	47.39	10.43	33.27
<i>General Models</i>									
Qwen2.5-VL-7B-Instruct	89.52	76.70	83.73	91.92	79.60	86.56	29.38	7.12	20.87
w/ GUI-RCPO	92.68	78.61	86.32	94.85	82.13	89.31	36.75	8.44	25.93
w/ GUI-SVPO	92.54	79.65	86.71	95.40	83.21	90.09	35.41	8.61	25.17
w/ GUI-RCPO-RC	92.11	79.48	86.40	94.71	82.49	89.39	36.54	9.11	26.06
w/ GUI-SVPO-SV	94.12	81.22	88.29	96.52	85.92	91.90	42.48	10.60	30.30
<i>GUI-specific Models</i>									
UI-TARS-1.5-7B	92.54	82.09	87.81	95.26	84.29	90.49	54.48	16.50	39.97
w/ GUI-RCPO	92.40	84.87	88.99	95.54	85.02	90.96	57.11	18.38	42.31
w/ GUI-SVPO	92.40	85.04	89.07	95.12	85.92	91.12	59.78	18.21	43.90
w/ GUI-RCPO-RC	92.68	82.96	89.29	95.13	86.44	91.35	54.86	14.90	39.60
w/ GUI-SVPO-SV	94.26	84.87	90.02	96.52	87.19	92.45	58.49	16.92	42.61

**Fig. 3:** Accuracy (%) across training steps of Qwen2.5-VL-3B-Instruct and Qwen2.5-VL-7B-Instruct throughout GUI-SVPO.

Sampling temperature. As illustrated in Fig. 2, grounding performance is highly sensitive to the decoding temperature. A moderate temperature of around 0.5 achieves the optimal balance between output diversity and coordinate stability. Lower temperatures restrict spatial exploration across multi-scale views, while higher temperatures introduce excessive spatial noise that disrupts the consensus voting process.

Rollout budget N . For a given budget N , rollouts are distributed evenly across the original and scaled views. Performance climbs rapidly as N increases from 4 to 32, and plateaus around $N = 64$. For instance, the accuracy of Qwen2.5-VL-3B improves significantly from 80.66% at $N = 4$ to 84.67% at $N = 64$. Scaling further to $N = 128$ yields diminishing returns, indicating that 64 samples provide sufficient candidate density for reliable spatial voting.

Table 4: Cross-benchmark generalization results. Models are trained on ScreenSpot-v2 and evaluated on ScreenSpot (SS) and ScreenSpot-Pro (SSpro).

Model	SS			SSpro			
	Text Avg.	Icon Avg.	Avg.	Text Avg.	Icon Avg.	Avg.	
<i>General Models</i>							
Qwen2.5-VL-3B-Instruct	88.23	63.65	77.12	30.30	5.63	20.87	
w/ GUI-RCPO	93.00	73.18	83.96	41.56	8.94	29.10	
w/ GUI-SVPO	93.58	73.43	84.36	42.15	9.54	29.84	
w/ GUI-RCPO-RC	93.15	73.66	84.27	41.45	9.93	29.41	
w/ GUI-SVPO-SV	93.97	75.36	85.45	46.51	11.09	32.98	
<i>General Models</i>							
Qwen2.5-VL-7B-Instruct	89.52	76.70	83.73	29.38	7.12	20.87	
w/ GUI-RCPO	92.68	79.12	86.55	36.44	8.28	25.68	
w/ GUI-SVPO	93.11	80.18	87.26	35.82	8.44	25.36	
w/ GUI-RCPO-RC	92.54	79.31	86.55	36.13	8.94	25.74	
w/ GUI-SVPO-SV	93.54	81.92	88.29	41.56	11.92	30.23	
<i>GUI-specific Models</i>							
UI-TARS-1.5-7B	92.54	82.09	87.82	54.48	16.50	39.97	
w/ GUI-RCPO	92.39	82.78	88.05	52.51	17.34	39.07	
w/ GUI-SVPO	92.40	84.17	88.68	57.12	16.39	41.56	
w/ GUI-RCPO-RC	93.11	83.30	88.68	55.17	15.73	40.10	
w/ GUI-SVPO-SV	93.97	86.44	90.57	57.52	16.56	41.87	

Expansion size α . For models predicting point coordinates, α defines the bounding box size used to simulate the model’s visual attention region. As shown in Fig. 2, accuracy follows a bell-shaped curve, peaking between $\alpha = 50$ and $\alpha = 70$. An overly small α underestimates the target boundary, limiting spatial overlap during multi-view voting. Conversely, an excessively large α incorporates irrelevant surrounding context into the consensus grid, thereby deviating the predicted center from the actual UI component.

Augmentation strategies. We further evaluate the impact of scaling configurations as shown in Table 5. Relying on a single scaled view is sub-optimal (e.g., 84.28% with a 1.2 factor and 83.73% with a 0.8 factor). Combining zoom-in and zoom-out perspectives boosts accuracy to 84.67%, while employing seven distinct isotropic scaling factors yields consistent gains up to 85.77%, demonstrating that finer scale diversity enriches spatial invariance. Conversely, anisotropic re-sizing (scaling horizontal and vertical dimensions differently) causes a noticeable performance drop. This indicates that maintaining isotropic proportions during multi-scale transformation is essential to preserve the structural features of UI elements for accurate grounding.

Training dynamics and model capacity. As shown in Fig. 3, we track the evaluation accuracy of Qwen2.5-VL-3B-Instruct and Qwen2.5-VL-7B-Instruct across training steps to analyze the learning dynamics of our approach. Both models exhibit a steep performance climb within the first 30 to 50 steps across all benchmarks, indicating that our multi-scale voting mechanism provides dense, high-quality pseudo-labels for rapid policy optimization. In the later stages of training, the trajectories diverge due to model capacity differences. The 7B model

Table 5: Ablation study on scaling factors for GUI-SV on ScreenSpot-v2 with Qwen2.5-VL-3B-Instruct. We compare square scaling (isotropic zoom-in/out) and non-square scaling (anisotropic resizing).

Scaling Factors	Text Avg.	Icon Avg.	Avg.
<i>Square Scaling</i>			
1.2	93.04	72.93	84.28
0.8	93.03	71.66	83.73
1.2 0.8 (default)	93.31	73.47	84.67
1.1 0.9	92.76	72.74	84.04
1.3 0.7	94.01	72.56	84.67
1.4 0.6	93.73	72.75	84.59
1.5 0.5	93.73	71.84	84.20
1.1 1.2 0.8	92.76	74.55	84.83
1.1 1.2 1.3 0.9 0.8	93.87	74.37	85.38
1.1 1.2 1.3 0.9 0.8	92.90	74.73	84.98
1.1 1.2 1.3 0.9 0.8 0.7	93.46	74.55	85.22
1.1 1.2 1.3 1.4 0.9 0.8 0.7	93.59	74.91	85.46
1.1 1.2 1.3 1.4 0.9 0.8 0.7 0.6	93.73	75.45	85.77
<i>Non-square Scaling</i>			
1.2–0.8 1.2–0.8	91.78	72.20	83.25
0.8–1.2 0.8–1.2	93.03	70.76	83.33

Table 6: Comparison of data augmentation methods on ScreenSpot-v2 with Qwen2.5-VL-3B-Instruct.

Method	Text Avg.	Icon Avg.	Avg.
Scaling	93.31	73.47	84.67
Crop	91.50	71.12	82.63
Style Augmentation	93.59	70.22	83.41

demonstrates robust stability and maintains a high plateau even on the challenging out-of-distribution ScreenSpot-Pro benchmark, whereas the 3B model peaks earlier and exhibits a slight degradation. This suggests that while our method effectively bootstraps grounding capabilities, higher-capacity models are better equipped to absorb spatial consensus rewards and generalize without overfitting to the training distribution.

Comparison with alternative augmentations. We compare our scale approach against **Crop augmentation**, which upsamples a local region at the expense of global context, and **Style augmentation**, which applies pixel-level visual transformations while preserving geometry. As shown in Table 6, our multi-scale method achieves the best overall performance. Crop augmentation yields the lowest accuracy, indicating that the loss of global layout cues severely

impairs the model’s spatial reasoning. Meanwhile, Style augmentation remains competitive for text targets but noticeably degrades icon localization, as GUI icons rely heavily on their original color and style semantics. Overall, our scale augmentation optimally balances global context and visual feature preservation, providing the most robust pseudo-labels.

5 Conclusion

This paper introduces GUI-SV, a multi-scale spatial voting method for test-time GUI grounding, along with its RL variant GUI-SVPO. By rescaling the full image across multiple resolutions and aggregating predictions via voting, we address the correlated-error limitation of single-scale approaches. Extensive experiments show state-of-the-art performance, with GUI-SVPO-SV boosting ScreenSpot-Pro accuracy from 20.87% to 33.27% without human annotations. Unlike cropping-based strategies, our full-image rescaling preserves global layout context. These results highlight scale variation as a critical factor in GUI grounding, offering a practical direction for building generalizable, label-efficient GUI agents.

References

1. Bai, S., Cai, Y., Chen, R., Chen, K., Chen, X., Cheng, Z., Deng, L., Ding, W., Gao, C., Ge, C., et al.: Qwen3-vl technical report. arXiv preprint arXiv:2511.21631 (2025)
2. Chen, C., Shao, J., Lu, D., Hu, H., Liu, X., Yao, H., Liu, W.: Gui-eyes: Tool-augmented perception for visual grounding in gui agents. In: Proc. AAAI. pp. 29350–29358 (2026)
3. Chen, L., Zhou, H., Zhang, J., Tong, P., Zhang, X., Kong, Q., Liu, C., Liu, Y., Wang, W., Wang, Y., et al.: Ui-ins: Enhancing gui grounding with multi-perspective instruction as reasoning. In: Proc. ICLR. pp. 54418–54444 (2026)
4. Cheng, K., Sun, Q., Chu, Y., Xu, F., YanTao, L., Zhang, J., Wu, Z.: SeeClick: Harnessing gui grounding for advanced visual gui agents. In: Proc. ACL. pp. 9313–9332 (2024)
5. Du, Y., Yan, Y., Tang, F., Lu, Z., Zong, C., Lu, W., Jiang, S., Shen, Y.: Test-time reinforcement learning for gui grounding via region consistency. In: Proc. AAAI. pp. 30593–30601 (2026)
6. Gou, B., Wang, R., Zheng, B., Xie, Y., Chang, C., Shu, Y., Sun, H., Su, Y.: Navigating the digital world as humans do: Universal visual grounding for gui agents. In: Proc. ICLR (2024)
7. Gu, Z., Zeng, Z., Xu, Z., Zhou, X., Shen, S., Liu, Y., Zhou, B., Meng, C., Xia, T., Chen, W., et al.: Ui-venus technical report: Building high-performance ui agents with rft. arXiv preprint arXiv:2508.10833 (2025)
8. Hong, W., Wang, W., Lv, Q., Xu, J., Yu, W., Ji, J., Wang, Y., Wang, Z., Dong, Y., Ding, M., et al.: Cogagent: A visual language model for gui agents. In: Proc. CVPR. pp. 14281–14290 (2024)
9. Hsieh, Z., Yang, S., Wei, T.J.: Zonui-3b: Competitive gui grounding with a 3b vlm trained on a single consumer gpu. pp. 959–966 (2026)

10. Jiang, Z., Xie, S., Li, W., Zu, W., Li, P., Qiu, J., Pei, S., Ma, L., Huang, T., Wang, M., et al.: Zoom in, click out: Unlocking and evaluating the potential of zooming for gui grounding. *arXiv preprint arXiv:2512.05941* (2025)
11. Kang, W., Lei, B., Liu, G., Ding, C., Yan, Y.: Guirlyg: Incentivize gui visual grounding via empirical exploration on reinforcement learning. In: *Proc. ICLR*. pp. 63861–63875 (2026)
12. Li, A.Y., Yu, B., Lei, D., Ren, T., Liu, S.: Chain-of-ground: Improving gui grounding via iterative reasoning and reference feedback. *arXiv preprint arXiv:2512.01979* (2025)
13. Li, K., Meng, Z., Lin, H., Luo, Z., Tian, Y., Ma, J., Huang, Z., Chua, T.S.: Screenspot-pro: Gui grounding for professional high-resolution computer use. In: *Proc. ACM-MM* (2025)
14. Lin, K.Q., Li, L., Gao, D., Yang, Z., Wu, S., Bai, Z., Lei, S.W., Wang, L., Shou, M.Z.: Showui: One vision-language-action model for gui visual agent. In: *Proc. CVPR*. pp. 19498–19508 (2025)
15. Liu, Y., Li, P., Wei, Z., Xie, C., Hu, X., Xu, X., Zhang, S., Han, X., Yang, H., Wu, F.: Infiguiagent: A multimodal generalist gui agent with native reasoning and reflection. pp. 1035–1051 (2026)
16. Liu, Y., Li, P., Xie, C., Hu, X., Han, X., Zhang, S., Yang, H., Wu, F.: Infigui-r1: Advancing multimodal gui agents from reactive actors to deliberative reasoners. *arXiv preprint arXiv:2504.14239* (2025)
17. Liu, Z., Feng, T., Kang, B., Yang, Y., Luo, J.: Zoom to essence: Trainless gui grounding by inferring upon interface elements. *arXiv preprint arXiv:2603.14448* (2026)
18. Lu, Z., Chai, Y., Guo, Y., Yin, X., Liu, L., Wang, H., Xiao, H., Ren, S., Zhao, P., Liu, G., et al.: Ui-r1: Enhancing efficient action prediction of gui agents by reinforcement learning. In: *Proc. AAAI*. pp. 17608–17616 (2026)
19. Luo, R., Wang, L., He, W., Chen, L., Li, J., Xia, X.: Gui-r1: A generalist r1-style vision-language action model for gui agents. *arXiv preprint arXiv:2504.10458* (2025)
20. Luo, T., Logeswaran, L., Johnson, J., Lee, H.: Visual test-time scaling for gui agent grounding. In: *Proc. ICCV*. pp. 19989–19998 (2025)
21. Muennighoff, N., Yang, Z., Shi, W., Li, X.L., Fei-Fei, L., Hajishirzi, H., Zettlemoyer, L., Liang, P., Candès, E., Hashimoto, T.B.: s1: Simple test-time scaling. In: *Proc. EMNLP*. pp. 20286–20332 (2025)
22. Nguyen, A.: Improved gui grounding via iterative narrowing. *arXiv preprint arXiv:2411.13591* (2024)
23. Pei, S., Tang, L., Duan, T., Chen, L., Li, S., Huang, K., Jing, Y., Yan, Y., Zhang, B., Jiang, C., et al.: Adazoom-gui: Adaptive zoom-based gui grounding with instruction refinement. *arXiv preprint arXiv:2603.17441* (2026)
24. Qian, R., Yin, X., Deng, C., Peng, Z., Xiong, J., Zhai, W., Dou, D.: Uground: Towards unified visual grounding with unrolled transformers. *arXiv preprint arXiv:2510.03853* (2025)
25. Qin, Y., Ye, Y., Fang, J., Wang, H., Liang, S., Tian, S., Zhang, J., Li, J., Li, Y., Huang, S., et al.: Ui-tars: Pioneering automated gui interaction with native agents. *arXiv preprint arXiv:2501.12326* (2025)
26. Seed, B.: Ui-tars-1.5. <https://seed-tars.com/1.5> (2025)
27. Shen, H., Liu, P., Li, J., Fang, C., Ma, Y., Liao, J., Shen, Q., Zhang, Z., Zhao, K., Zhang, Q., et al.: Vlm-r1: A stable and generalizable r1-style large vision-language model. *arXiv preprint arXiv:2504.07615* (2025)

28. Snell, C., Lee, J., Xu, K., Kumar, A.: Scaling llm test-time compute optimally can be more effective than scaling model parameters. arXiv preprint arXiv:2408.03314 (2024)
29. Tang, F., Chen, B., Lu, Z., Chen, T., Nong, S., Jiang, T., Xu, W., Lu, W., Xiao, J., Zhuang, Y., et al.: Ui-zoomer: Uncertainty-driven adaptive zoom-in for gui grounding. arXiv preprint arXiv:2604.14113 (2026)
30. Tang, F., Gu, Z., Lu, Z., Liu, X., Shen, S., Meng, C., Wang, W., Zhang, W., Shen, Y., Lu, W., et al.: Gui-g²: Gaussian reward modeling for gui grounding. In: Proceedings of the AAAI Conference on Artificial Intelligence (2026)
31. Wang, H., Zou, H., Song, H., Feng, J., Fang, J., Lu, J., Liu, L., Luo, Q., Liang, S., Huang, S., et al.: Ui-tars-2 technical report: Advancing gui agent with multi-turn reinforcement learning. arXiv preprint arXiv:2509.02544 (2025)
32. Wang, S., Liu, W., Chen, J., Zhou, Y., Gan, W., Zeng, X., Che, Y., Yu, S., Hao, X., Shao, K., et al.: Gui agents with foundation models: A comprehensive survey. arXiv preprint arXiv:2411.04890 (2024)
33. Wang, X., Wang, B., Lu, D., Yang, J., Xie, T., Wang, J., Deng, J., Guo, X., Xu, Y., Wu, C., et al.: Opencua: Open foundations for computer-use agents. pp. 139756–139806 (2025)
34. Wu, H., Chen, H., Cai, Y., Liu, C., Ye, Q., Yang, M.H., Wang, Y.: Dimo-gui: Advancing test-time scaling in gui grounding via modality-aware visual reasoning. In: Proc. EMNLP. pp. 26246–26256 (2025)
35. Wu, Q., Cheng, K., Yang, R., Zhang, C., Yang, J., Jiang, H., Mu, J., Peng, B., Qiao, B., Tan, R., et al.: Gui-actor: Coordinate-free visual grounding for gui agents. In: Proc. NeurIPS. pp. 15101–15128 (2025)
36. Wu, Z., Wu, Z., Xu, F., Wang, Y., Sun, Q., Jia, C., Cheng, K., Ding, Z., Chen, L., Liang, P.P., et al.: Os-atlas: Foundation action model for generalist gui agents. In: Proc. ICLR. pp. 5090–5108 (2025)
37. Xu, H., Zhang, X., Liu, H., Wang, J., Zhu, Z., Zhou, S., Hu, X., Gao, F., Cao, J., Wang, Z., et al.: Mobile-agent-v3. 5: Multi-platform fundamental gui agents. arXiv preprint arXiv:2602.16855 (2026)
38. Xue, T., Peng, C., Huang, M., Guo, L., Han, T., Wang, H., Wang, J., Zhang, X., Yang, X., Zhao, D., et al.: Evocua: Evolving computer use agents via learning from scalable synthetic experience. arXiv preprint arXiv:2601.15876 (2026)
39. Yang, Y., Wang, Y., Li, D., Luo, Z., Chen, B., Huang, C., Li, J.: Aria-ui: Visual grounding for gui instructions. pp. 22418–22433 (2025)
40. Yuan, X., Zhang, J., Li, K., Cai, Z., Yao, L., Chen, J., Wang, E., Hou, Q., Chen, J., Jiang, P.T., et al.: Se-gui: Enhancing visual grounding for gui agents via self-evolutionary reinforcement learning. In: Proc. NeurIPS. pp. 127658–127679 (2025)
41. Zhang, Q., Lyu, F., Sun, Z., Wang, L., Zhang, W., Hua, W., Wu, H., Guo, Z., Wang, Y., Muennighoff, N., et al.: A survey on test-time scaling in large language models: What, how, where, and how well? arXiv preprint arXiv:2503.24235 (2025)
42. Zhang, Y., Pan, Z., Zeng, Z., Shen, S., Meng, C., Zhu, L.: Mvp: Multiple view prediction improves gui grounding. In: Proc. CVPR. pp. 27482–27492 (2026)
43. Zhao, Z., Liu, Y., Liu, Y., Wang, H., Tian, L., Zhou, X., You, Y., Yu, Z., Yu, Y., Zhou, J.: Points-gui-g: Gui-grounding journey. arXiv preprint arXiv:2602.06391 (2026)
44. Zhou, H., Zhang, X., Tong, P., Zhang, J., Chen, L., Kong, Q., Cai, C., Liu, C., Wang, Y., Zhou, J., et al.: Mai-ui technical report: Real-world centric foundation gui agents. arXiv preprint arXiv:2512.22047 (2025)

- 45. Zhu, J., Wang, W., Chen, Z., Liu, Z., Ye, S., Gu, L., Tian, H., Duan, Y., Su, W., Shao, J., et al.: Internvl3: Exploring advanced training and test-time recipes for open-source multimodal models. arXiv preprint arXiv:2504.10479 (2025)
- 46. Zuo, Y., Zhang, K., Sheng, L., Qu, S., Cui, G., Zhu, X., Li, H., Long, X., Hua, E., Qi, B., et al.: Ttrl: Test-time reinforcement learning. In: Proc. NeurIPS. pp. 131459–131483 (2025)